Mergesort

Divide, sort, merge – the N log N sort

Marcus Birkenkrahe

COSC 2230 · CS III · Augustana University

Fall 2026

By the end you can

1. explain mergesort: divide, sort each half recursively, merge
2. implement `merge()` with an auxiliary array and trace it with `i`, `j`, `k`
3. implement the recursive `sort(a, lo, hi)` and trace its calls
4. argue that mergesort is correct
5. derive its cost: $2N \lg N$ data moves, $\frac{1}{2} N \lg N$ to $N \lg N$ compares
6. confirm $N \log N$ growth with a doubling test – mergesort scales
7. name its price, an extra array, and reduce other problems to sorting

1,000 exam papers, two friends

How do you split the work?

Each friend sorts half – then you merge

Look at the top paper of each sorted pile, take the one that comes first, repeat.

With insertion sort, how long does half the pile take? → A quarter of the time: $(N/2)^2 = N^2/4$.
→ Two halves: half the time. Merging: one look per paper.

Now do the same for each half again, and again . . .

Mergesort: the idea

- **Divide** the array into two halves
- **Sort** each half – recursively
- **Merge** the two sorted halves into a sorted whole
- Base case: 0 or 1 entries are sorted
- **Divide-and-conquer**: independent parts, solved independently, then combined

input

```
was had him and | you his the but
```

sort left, sort right

```
and had him was | but his the you
```

merge

```
and but had him his the was you
```

John von Neumann (1903–1957) wrote mergesort in 1945, to test a new computer on a task other than numerical calculation.

Merging needs an extra array

- Merging two sorted halves inside the same array (**in place**) is very hard
- So `merge()` writes into an **auxiliary array** `aux[]` – and copies it back
- From outside it looks as if the merge happened in place
- The price: an extra array of size N (quicksort and heapsort sort in place – COSC 3210)

Half-open intervals, as in binary search

`a[lo, mid)` = `a[lo]` ... `a[mid-1]` `a[mid, hi)` = `a[mid]` ... `a[hi-1]`

Code-along 1: `Merge.java` in five steps

`Merge.java`

1	`private static String[] aux;`	the extra array
2	`merge(a, lo, mid, hi)`	fill aux[0, N), copy back
3	`sort(a)`	allocate aux once, start
4	`sort(a, lo, hi)`	base case, two calls, merge
5	`main()`	test client from Insertion

merge: two sorted halves
had was | and him
→ and had him was

sort: split, sort each half, merge
first run after step 5

```
$ java Merge < names16.txt   →   alice bob carlos . . . wendy
```

Later: `TestMerge.java` – the doubling test from `TestInsertion`, with `Merge.sort(a)`

Merge: merge()

→ **Switch to editor · code-along 1**

1. Create **`Merge.java`** with a static field **`String[] aux`**
2. Write **`merge(a, lo, mid, hi)`**: fill `aux[0, N)`, copy back

data: `birkenkrahe.github.io/data/names16.txt` · classpath: `.:stdlib.jar`

merge(): one index per half, one into aux[]

```
private static String[] aux;
public static void merge(String[] a,
                         int lo, int mid, int hi)
{
    int i = lo, j = mid, N = hi - lo;
    for (int k = 0; k < N; k++)
    {
        if      (i == mid) aux[k] = a[j++];
        else if (j == hi)  aux[k] = a[i++];
        else if (a[j].compareTo(a[i]) < 0)
                           aux[k] = a[j++];
        else               aux[k] = a[i++];
    }
    for (int k = 0; k < N; k++)
        a[lo + k] = aux[k];
}
```

- i: left half
- j: right half
- k: next free place in aux
- left used up: take right
- right used up: take left
- else the smaller one
- then copy back

Trace: one merge, one row per step

			left half a[0, 4)				right half a[4, 8)				
i	j	k	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	aux[k]
0	4	0	**and**	had	him	was	**but**	his	the	you	and
1	4	1	and	**had**	him	was	**but**	his	the	you	but
1	5	2	and	**had**	him	was	but	**his**	the	you	had
2	5	3	and	had	**him**	was	but	**his**	the	you	him
3	5	4	and	had	him	**was**	but	**his**	the	you	his
3	6	5	and	had	him	**was**	but	his	**the**	you	the
3	7	6	and	had	him	**was**	but	his	the	**you**	was
4	7	7	and	had	him	was	but	his	the	**you**	you

bold: a[i] and a[j], compared **teal**: taken into aux[k] grey: already in aux

How many compares? → 7: none in the last row – the left half is used up.
How many data moves? → 8 into aux, 8 back: $2N = 16$.

companion card 1: birkenkrahe.github.io/demos/merge-sort.html

Warm-up: trace one merge

WARM-UP · 5 MIN · CLOSED BOOK

`merge(a, 0, 4, 8)` merges the two sorted halves:

```
carol eve oscar wendy   |  bob dave frank peggy
```

1. One row per step: **i, j, k** and the entry that goes into `aux[k]`
2. Count the **compares**

Which case of `merge()` takes the last entry?

Solution

i	j	k	aux[k]
0	4	0	bob
0	5	1	carol
1	5	2	dave
1	6	3	eve
2	6	4	frank
2	7	5	oscar
3	7	6	peggy
3	8	7	wendy

- The halves alternate: 7 compares, one for every entry but the last
- After peggy, `j == hi`: the **right** half is used up, and wendy goes over without a compare
- In the class example the **left** half ran out first

Merge: `sort()` and the test client

→ **Switch to editor · code-along 1**

3. Write **`sort(a)`**: allocate aux once, call `sort(a, 0, a.length)`
4. Write **`sort(a, lo, hi)`**: base case, `mid`, two calls, merge
5. Test client: the one from `Insertion` – run it on **`names16.txt`**

data: `birkenkrahe.github.io/data/names16.txt` · classpath: `.:stdlib.jar`

sort(): allocate once, recurse

```
public static void sort(String[] a)
{
    aux = new String[a.length];
    sort(a, 0, a.length);
}

public static void sort(String[] a,
                        int lo, int hi)
{
    int N = hi - lo;
    if (N <= 1) return;
    int mid = lo + N/2;
    sort(a, lo, mid);
    sort(a, mid, hi);
    merge(a, lo, mid, hi);
}
```

```
$ java Merge < names16.txt
alice bob carlos carol craig
dave erin eve frank oscar
peggy trent trudy victor
walter wendy
```

aux is a field: `null` until `sort(a)` assigns it. With `String[] aux = ...` in `sort(a)`, a local variable hides the field, and the first `merge()` throws a **`NullPointerException`**.

The same algorithm works for `Comparable[]`, with aux passed as a parameter.

Overloading: same name, different parameters

- Two methods named `sort()`: Java picks the one whose parameters match – as with `StdOut.println()`
- You know the pattern: `sumAcc(n)` calls the **private** helper `sumAcc(n, acc)`, which is only correct when `acc` starts at 0
- Same name **and** same parameter types: not allowed

```
$ javac -cp .:stdlib.jar Merge.java
error: method sort(String[],int,int) is already defined in class Merge
```

Code-along: paste a second copy of `sort(a, lo, hi)` into `Merge.java`, compile, delete it

Questions about the code

Why allocate `aux` in `sort(a)`, not in `merge()`?
→ $N - 1$ merges × a new array of length N: about N^2 steps.

Why `< 0` and not `<= 0`?
→ On a tie the left entry goes first: equal keys keep their order (**stable**).

What happens without `if (N <= 1) return;`?
→ `sort(a, lo, lo+1)` calls itself again: `StackOverflowError`.

Why check `i == mid` and `j == hi` first?
→ A used-up half has no entry to compare: `a[hi]` may be outside the array.

companion card 1, "equal keys": `birkenkrahe.github.io/demos/merge-sort.html`

Trace: the calls of `sort()`

	0	1	2	3	4	5	6	7
	was	had	him	and	you	his	the	but
`sort(a, 0, 8)`								
`sort(a, 0, 4)`								
`sort(a, 0, 2)`	had	was	him	and	you	his	the	but
`sort(a, 2, 4)`	had	was	and	him	you	his	the	but
`merge 0..4`	and	had	him	was	you	his	the	but
`sort(a, 4, 8)`								
`sort(a, 4, 6)`	and	had	him	was	his	you	the	but
`sort(a, 6, 8)`	and	had	him	was	his	you	but	the
`merge 4..8`	and	had	him	was	but	his	the	you
`merge 0..8`	and	but	had	him	his	the	was	you

How deep does the recursion get for 8 entries? For a million? → 8, 4, 2, 1: 4 calls. A million: $\lg 10^6 + 1 \approx 21$.

Rows: the array after a call returns (sizes 0 and 1 not shown)

companion card 2: `birkenkrahe.github.io/demos/merge-sort.html`

Correctness: the three pieces

Piece	Guarantees	In `Merge.sort(a, lo, hi)`
base case	smallest cases right	$N \leq 1$: 0 or 1 entries are sorted
termination	an answer is reached	both halves are smaller than N
reduction	the answer is right	two sorted halves $\rightarrow$ `merge()` $\rightarrow$ sorted

Why `merge()` is right

Before each step, `aux[0, k)` holds the k smallest entries of both halves, in order. The next smallest is the smaller of `a[i]` and `a[j]`.

Odd N: `mid = lo + N/2` rounds down – the left half has one entry fewer ($N = 11$: 5 and 6).

companion card 2, "11 names (odd N)": `birkenkrahe.github.io/demos/merge-sort.html`

Practice: count the compares

PRACTICE · 5 MIN · CLOSED BOOK

Merge the two sorted halves with `merge()`:

(a)	alice	bob	carol	dave	erin	eve	frank	oscar
(b)	alice	carol	erin	frank	bob	dave	eve	oscar

1. Write the names in the order they go into `aux[]`
2. Count the **compares** of each merge

Same 8 names, same data moves. Why a different number of compares?

Solution

(a)

halves	alice	bob	carol	dave	erin	eve	frank	oscar	
aux[]	alice	bob	carol	dave	erin	eve	frank	oscar	
compares	1	1	1	1	–	–	–	–	**4**

`alice` to `dave` each win against `erin`; then the left half is used up (`i == mid`), and the rest goes over without a compare.

(b)

halves	alice	carol	erin	frank	bob	dave	eve	oscar	
aux[]	alice	bob	carol	dave	erin	eve	frank	oscar	
compares	1	1	1	1	1	1	1	–	**7**

The halves alternate: one compare for every name but the last.

Data moves: 16 in both, 8 into aux and 8 back.
One merge of N entries: between $N/2$ and $N - 1$ compares.

blue: left half amber: right half

Practice: trace mergesort by hand

PRACTICE · 10 MIN · CLOSED BOOK

`it was the best of times it was`

1. After each merge: write **lo, mid, hi** and the whole array
2. Count the **compares** of each merge
3. Add them up. How many data moves in total?
4. Check with **`MergeTrace`** (`java ... MergeTrace < itwas.txt`)

Where are the two `its` at the end – which one comes first?

Data: `birkenkrahe.github.io/data/itwas.txt`

Solution

lo	mid	hi	array after the merge								cmp
			it	was	the	best	of	times	it	was	
0	1	2	it	was	the	best	of	times	it	was	1
2	3	4	it	was	best	the	of	times	it	was	1
0	2	4	best	it	the	was	of	times	it	was	3
4	5	6	best	it	the	was	of	times	it	was	1
6	7	8	best	it	the	was	of	times	it	was	1
4	6	8	best	it	the	was	it	of	times	was	3
0	4	8	best	it	it	of	the	times	was	was	7

- 17 compares; data moves $2(2+2+4+2+2+4+8) = 48 = 2N \lg N$
- Two ties in the last merge: the left one goes first – the first `it` of the input stays first (stable)
- Three merges change nothing – but still cost a compare and 4 moves each

Interactive companion

birkenkrahe.github.io/demos/merge-sort.html

1. **Merge**: one move per click, with `i`, `j`, k and aux`[]` – try "equal keys"
2. **The whole recursion**: every call, base case and merge, with the call stack and the trace
3. **Cost per level**: $2N$ data moves on each of $\lg N$ levels
4. **Insertion sort against mergesort**: predicted times up to 10^{10}

Question: one split

Insertion sort on N random entries makes about $N^2/4$ compares.

Split once: insertion sort on each half, then one merge. How many compares now?

→ The two halves: $2 \cdot (N/2)^2/4 = N^2/8$. The merge: at most $N - 1$.

→ About $N^2/8 + N$: half the work. ($N = 1000$: 250,000 → about 126,000.)

Split the halves again? And again, down to size 1?

→ Each split halves the quadratic part and adds one level of merges, about N compares. At size 1 the quadratic part is gone: only the merges are left.

Question: data moves

A merge of two halves with 8 entries each: how many data moves?

A data move: one string copied from one array to the other.

→ 16 into aux, 16 back: $32 = 2N$.

The whole mergesort of the 16 names?

→ 1 merge of 16, 2 of 8, 4 of 4, 8 of 2: 32 moves per level, 4 levels – 128.

Does it depend on the order of the input?

→ No: every merge moves every entry there and back.

Insertion sort: 0 exchanges for sorted input, $N^2/2$ for reversed.

Analysis: count the data moves

Cost model: **data moves**. Exact for $N = 2^n$, so $n = \lg N$.

level	subarrays	size	data moves
1	1	N	$2N$
2	2	$N/2$	$2N$
3	4	$N/4$	$2N$
...	...	...	...
n	2^{n-1}	2	$2N$

Total

$\lg N$ levels $\times$ $2N = 2N \lg N$ data moves

$\lg$ = logarithm to base 2.

Counting the inner loop only gives $N \lg N$

companion card 3: `birkenkrahe.github.io/demos/merge-sort.html`

Why $N = 2^n$?

Why analyze $N = 2^n$ only?

- halving divides evenly: subarrays $N, N/2, N/4, \ldots, 2$
- the number of levels is a whole number: $n = \lg N$
- the count is exact: $2N \lg N$, no rounding

Binary search did the same:
$T(n) = T(n/2) + 1$ for $n = 2^k$.

Does it hold for $N = 1000$?
$\rightarrow$ 10 levels, the last one not full:
19,952 data moves.
$\rightarrow 2N \lg N = 19{,}932$ – 0.1% apart.

Every N

$2^n \leq N < 2^{n+1}$, and the cost never decreases: it lies between the costs for 2^n and 2^{n+1} – the same $N \log N$.

companion card 3, $N = 1000$: `birkenkrahe.github.io/demos/merge-sort.html`

Compares: $\frac{1}{2} N \lg N$ to $N \lg N$

- A merge of n entries: at most $n - 1$ compares, at least $n/2$ (one half used up first)
- So mergesort makes between $\frac{1}{2} N \lg N$ and $N \lg N$ compares

```
$ java MergeCount 4096
sorted   compares  24,576  moves  98,304   N lg N = 49,152   2N lg N = 98,304
random   compares  43,981  moves  98,304   N lg N = 49,152   2N lg N = 98,304
reversed compares  24,576  moves  98,304   N lg N = 49,152   2N lg N = 98,304
```

- Insertion sort on 4096 random strings: $N^2/4 \approx 4.2$ million compares
- **Not sensitive to input**: no bad case – and no good case either

companion card 3: birkenkrahe.github.io/demos/merge-sort.html

Question: predict

Mergesort sorts **1 million** random strings in **0.25 s**.

How long for **2 million**? For **16 million**?

→ Doubling N: a bit more than twice: $2 \lg(2N) / \lg N \approx 2.1$.
→ 16 million $= 2^4 \times$ 1 million: about $2.1^4 \approx 19$ times – about 5 s.

TestMerge: the doubling test

→ **Switch to editor · code-along 2**

1. Copy **`TestInsertion.java`** to **`TestMerge.java`**
2. Replace `Insertion.sort(a)` by **`Merge.sort(a)`**
3. Start where insertion sort stopped: **160,000** strings (27 s)
4. Double N up to 8 million – predict each time before you run

```
java Generator N 10 abc...z | java TestMerge
```

Read the table

N	Sedgewick		thelio	
	T_N (s)	ratio	T_N (s)	ratio
1 million	1		0.25	
2 million	2	2	0.76	3.0
4 million	5	2.5	1.69	2.2
8 million	10	2	3.87	2.3
16 million	20	2	8.45	2.2
...				
1 billion	1280	2		

Sedgewick (2016) · thelio: JDK 17, 2026-10-27

- The ratio settles at about 2: $N \log N$
- Exact for $N \log N$ at 16 million: $2 \times 24/23 = 2.09$
- Sedgewick: 1 billion strings in about 20 minutes
- Small runs are distorted (just-in-time compiler, cache)

Quadratic against linearithmic

N	insertion	mergesort
160,000	27 s	0.03 s
320,000	131 s	0.07 s
1 million	21 min	0.25 s
1 billion	41 years	11 min

same laptop; insertion from 1 million on and mergesort at 1 billion are predicted from the largest runs

New computer: $2\times$ speed, $2\times$ memory, so $2\times$ N. Time = doubling ratio / 2:

insertion	$4/2 = 2$	twice as long
mergesort	$\approx 2.1/2$	about the same

Mergesort scales.

The "quadratic-linearithmic chasm"

companion card 4: `birkenkrahe.github.io/demos/merge-sort.html`

Pitfall: an extra array in every call

```
public static void merge(String[] a, int lo, int mid, int hi)
{
    String[] aux = new String[a.length];
    ...
```

```
N=20000:  allocate once 0.00 s, allocate in every merge 0.19 s
N=40000:  allocate once 0.01 s, allocate in every merge 0.57 s
N=80000:  allocate once 0.02 s, allocate in every merge 1.64 s
N=160000: allocate once 0.03 s, allocate in every merge 5.56 s
```

Quadratic again

$N - 1$ merges, each filling an array of length N with `null`: about N^2 steps. Doubling ratio about 3 and rising toward 4. One line in the wrong place.

The price: memory – and the recursion depth

- **Memory**: `aux[]` is as large as the input. Insertion sort needs one temporary variable
- **Recursion depth**: at most $\lg N + 1$ calls on the stack – 5 for 16 names, 25 for 16 million
- No stack overflow (unlike `sum(20000)`): keep the recursion
- The loop version exists: **bottom-up mergesort** – merge sizes 1, 2, 4, ... with two loops (COSC 3210)

Mergesort in Java – and reduction to sorting

- **Stable**: equal keys keep their input order. Sort by name, then stably by grade – each grade stays in name order
- `Arrays.sort()` for objects: **TimSort**, a mergesort that finds sorted runs and sorts short runs with insertion sort – stable
- `Arrays.sort()` for `int[]`, `double[]`: a quicksort – in place, not stable
- **Reduction to sorting**: how would you solve it if the data were sorted? Often one linear pass does it
- Examples: are all values distinct? the median? remove duplicates? how often does each word occur?

Practice: word frequencies with your `Counter`

PRACTICE · 18 MIN · THREE STEPS

1. **DeDup**: print the words in sorted order, each once – equal words are neighbors after sorting.
   `DeDup < itwas.txt` → `best it of the times was`
2. **MergeComparable**: copy `Merge.java`, replace `String` by `Comparable` – as for `InsertionComparable`
3. **FrequencyCount**: one `Counter` per distinct word, `increment()` for every word; sort the counters; print the largest count first
   `FrequencyCount < itwas.txt` → `was 2, it 2, ...`

Why does the second sort work on `Counter` objects?

Your `Counter.java` from the `Comparable` code-along

Solution: FrequencyCount

```
String[] words = StdIn.readAllStrings();
MergeComparable.sort(words);
Counter[] zipf = new Counter[words.length];
int m = 0;
for (int i = 0; i < words.length; i++)
{
    if (i == 0 || !words[i].equals(words[i-1]))
        zipf[m++] = new Counter(words[i], 0);
    zipf[m-1].increment();
}
Counter[] counts = Arrays.copyOf(zipf, m);
MergeComparable.sort(counts);
for (int j = m-1; j >= 0; j--)
    StdOut.println(counts[j]);
```

- Two sorts: words (equal ones together), then counters (by count)
- `Counter` keeps the `Comparable` promise – the sort needs nothing else
- `copyOf`: `zipf[m, N)` is `null`
- Why was `2` before `it 2`?
- Cost: two sorts + one pass, $N \log N$

Wrap-up: mergesort

- Divide, sort each half recursively, merge
- `merge()`: auxiliary array, allocated once; ties go left – stable
- Correct: base case, smaller halves, correct merge
- $2N \lg N$ data moves, $\frac{1}{2} N \lg N$ to $N \lg N$ compares – for every input
- Doubling ratio about 2: mergesort scales
- The price: an extra array of size N
- Many problems reduce to sorting

Checkout question

Insertion sort is linear on sorted input – mergesort is not. Which one line in `sort(a, lo, hi)` makes it linear on sorted input, and still correct for every input?

Checkout answer

Before the merge in `sort(a, lo, hi)`:

```
if (a[mid-1].compareTo(a[mid]) <= 0) return;
```

- The largest entry on the left is not larger than the smallest on the right: `a[lo, hi)` is already sorted – skip the merge
- Sorted input: every merge skipped, $N - 1$ compares, no data moves – linear
- Random input: the check rarely succeeds – about N extra compares, still $N \log N$